

Pedestrian crossing detection is an important research direction in intelligent vehicles. This project, drawing on the application effects of YOLOv8 in engineering inspection and agricultural monitoring, proposes an improved YOLOv8 model that incorporates DCNv3. Although the YOLOv8 model has become a mainstream object detection model in the academic community due to its efficient inference speed and excellent detection accuracy, there are still many deficiencies in accurately applying the modified model to real-world scenarios. Therefore, our team has compiled a document. From environment preparation, dependency configuration to practical training, each step is accompanied by specific commands and tips to avoid pitfalls. Even those with no background can quickly get started. It covers the download of miniconda, cuda, cudnn, and pytorch, as well as the setup of corresponding version environments, to the use of YOLOv8 in PyCharm. By following the tutorial, excluding software download time, it can be configured and used within 90 minutes.

The entire process includes environmental configuration, model download, improvement research, and practical application, among other steps.

I. Pytorch Environmental Configuration	2
1. Python Working Environment	2
1.1 Miniconda	2
1.2 Pytorch	9
1.3 CUDNN	14
II YOLOv8	19
1. Installation of the basic model of yolov8	19
1.1 Pycharm	20
1.2 Download, verification, and usage of YOLOv8	24
1.3 Verify whether the GPU is available	29
III. Using GPU to Train YOLOv8 Model (Practical)	29
1. Improvement of the yolov8 model	29
1.1 Prepare the dataset	29
2.2 Initiate GPU training	30
3.3 Training via command line (more concise)	30

I. Pytorch Environmental Configuration

The operation of YOLOv8 requires users to configure a Pytorch working environment, so we first need to carry out a common Pytorch environmental configuration.

1. Python Working Environment

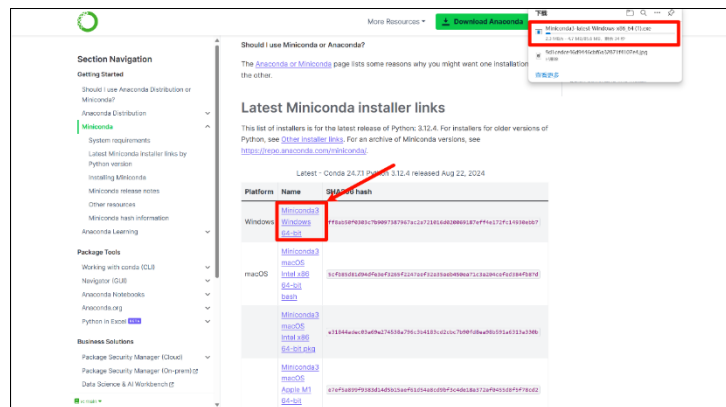
The setup of a Pytorch environment involves the installation of multiple software and environments, so the most important thing is the version compatibility among different software and their usability. This section will first guide you to install a Python environment download and management tool; then, we will help you download the corresponding versions of CUDA, cuDNN, and Pytorch to build a usable Pytorch environment.

1.1 Miniconda

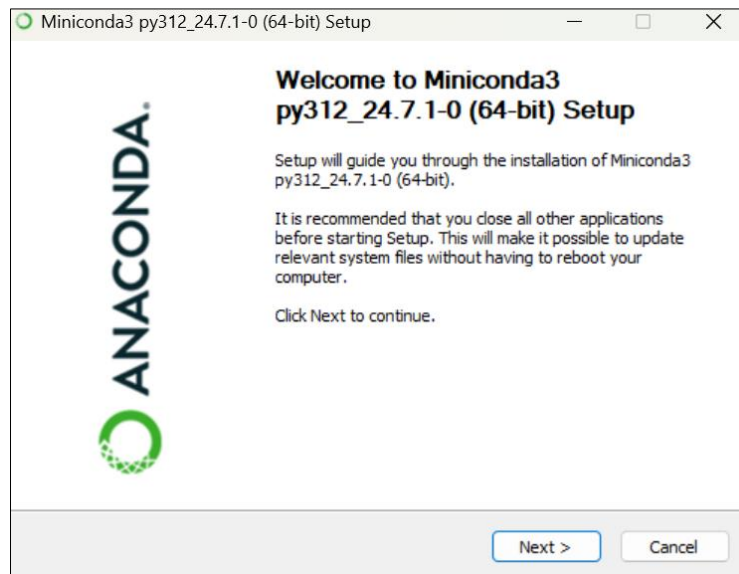
The team uses Miniconda for Python environment management. Miniconda is smaller than Anaconda and has sufficient functionality. It is recommended to learn Miniconda.

(1) Software Installation

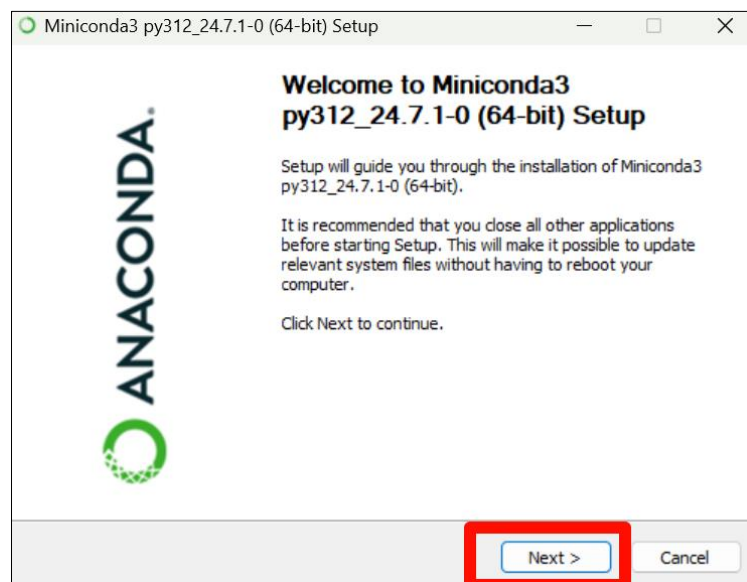
Input the following link: <https://www.anaconda.com/docs/getting-started/miniconda/main>, Click on the download link, and after opening the page, the software installation package will start downloading automatically.



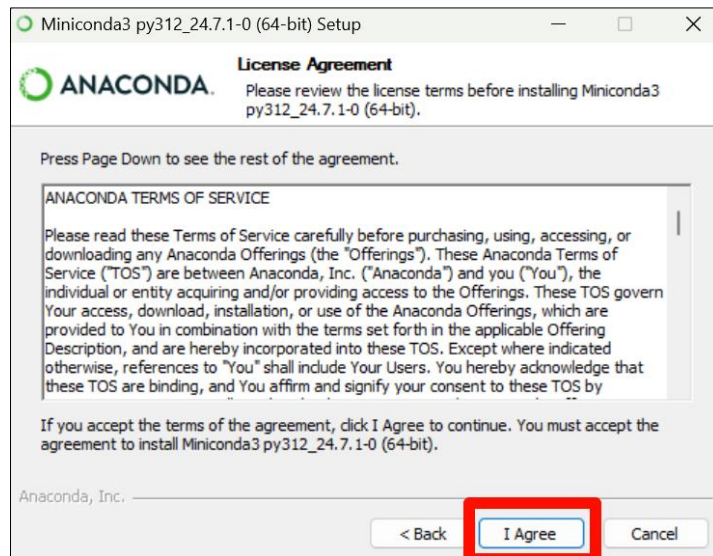
Click on the installation package, and this interface will pop up, then you can start downloading



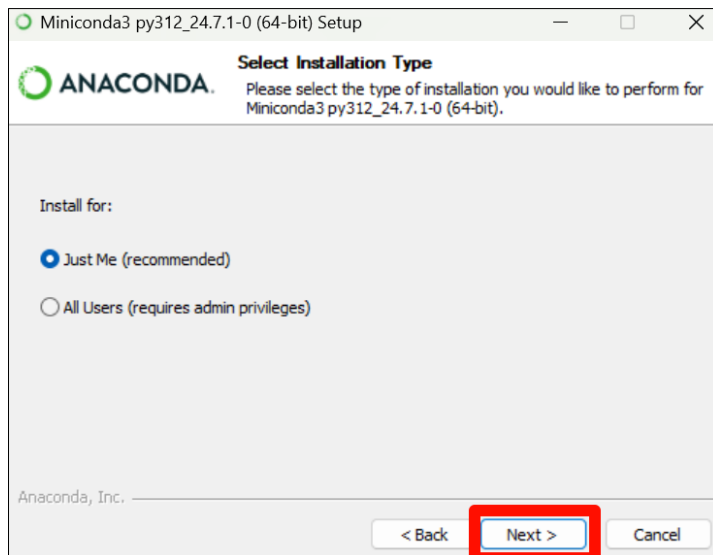
Click the "Next" button



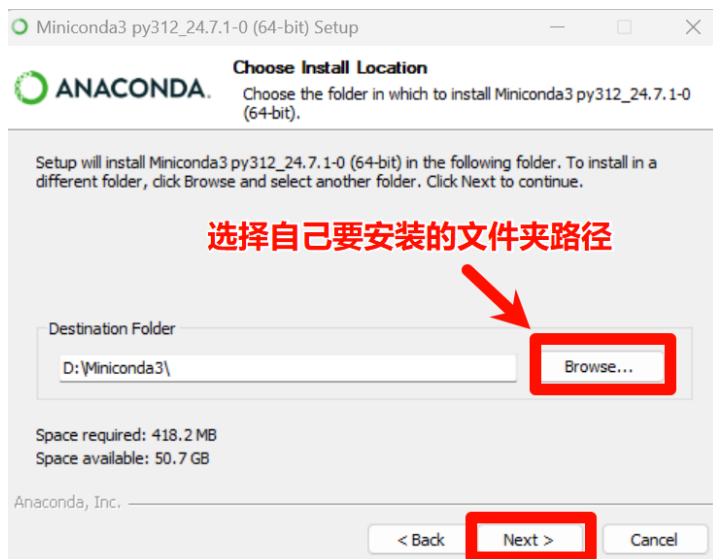
Click the " I Agree " button



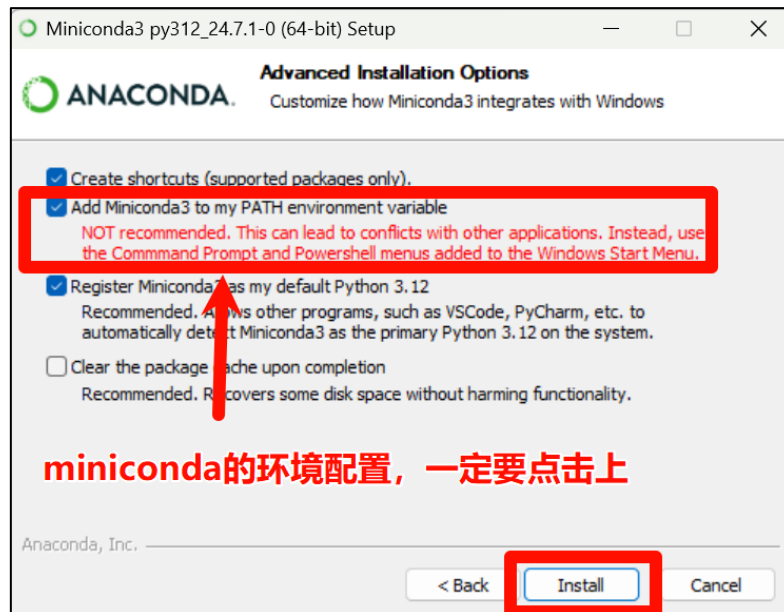
Select "Just Me", and then click the "Next" button



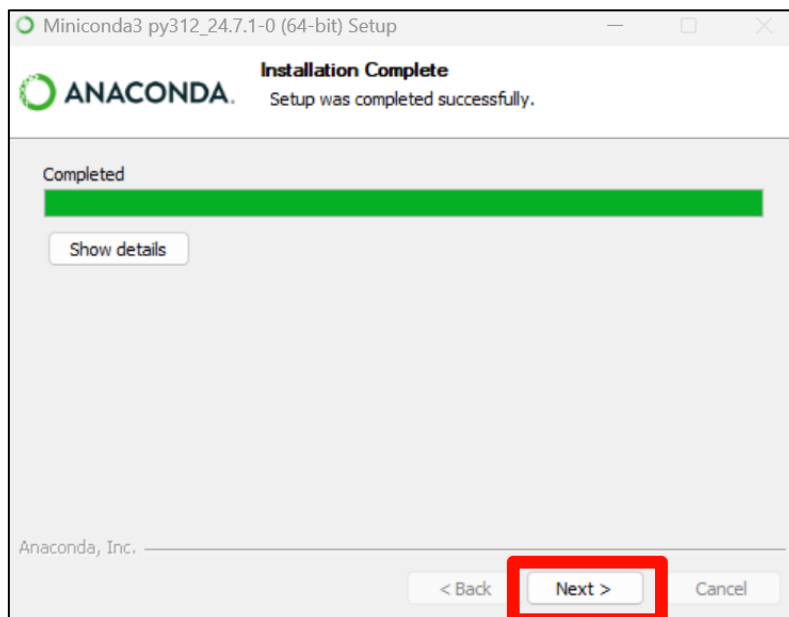
Click the "Browse" button and select the location of the file you want to install



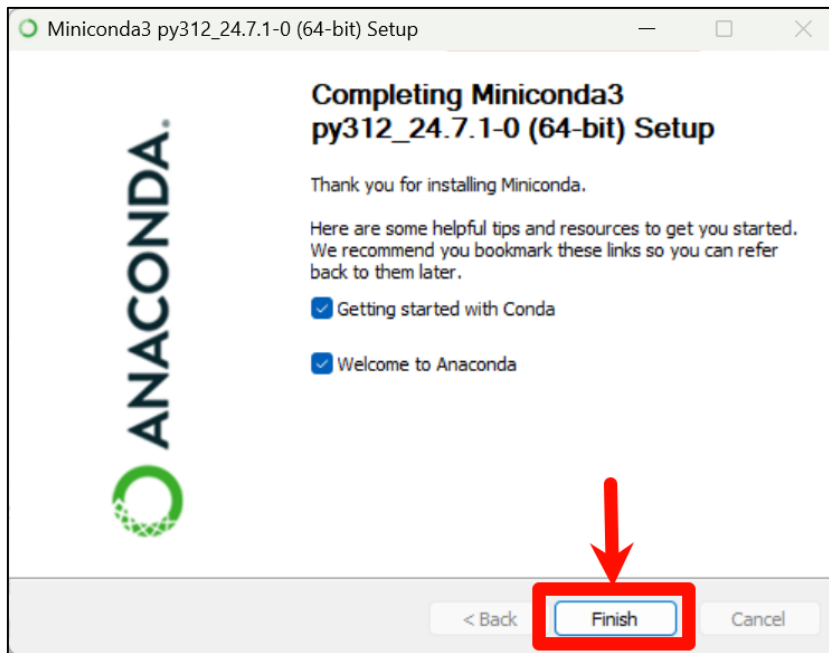
Check the items in the diagram, make sure to tick the PATH item, and click "Install" to start the software installation



Click the "Next" button

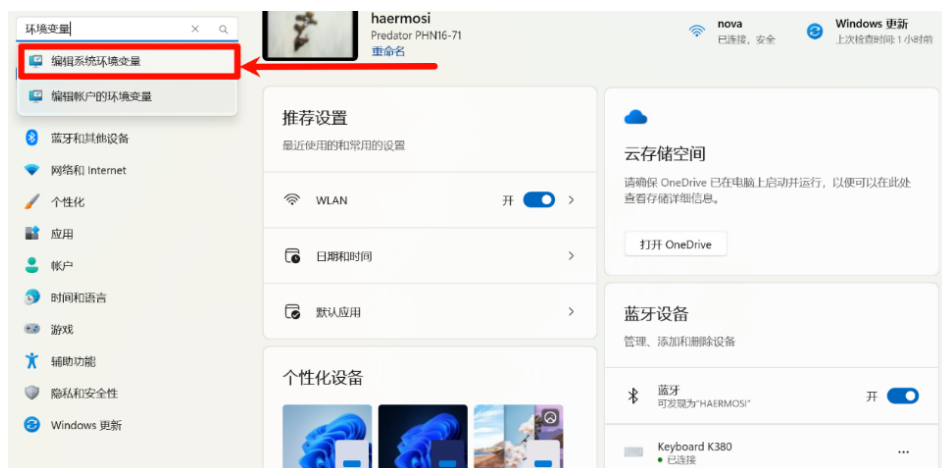


After the installation is complete, click the "Finish" button



(2) Environment Verification

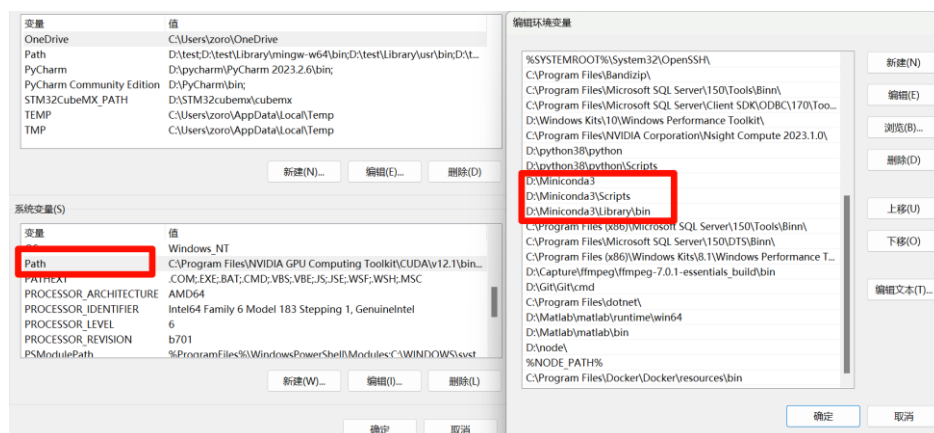
Input and open the environment variables in the computer settings (be sure to open it in the settings), and click "Edit system environment variables"



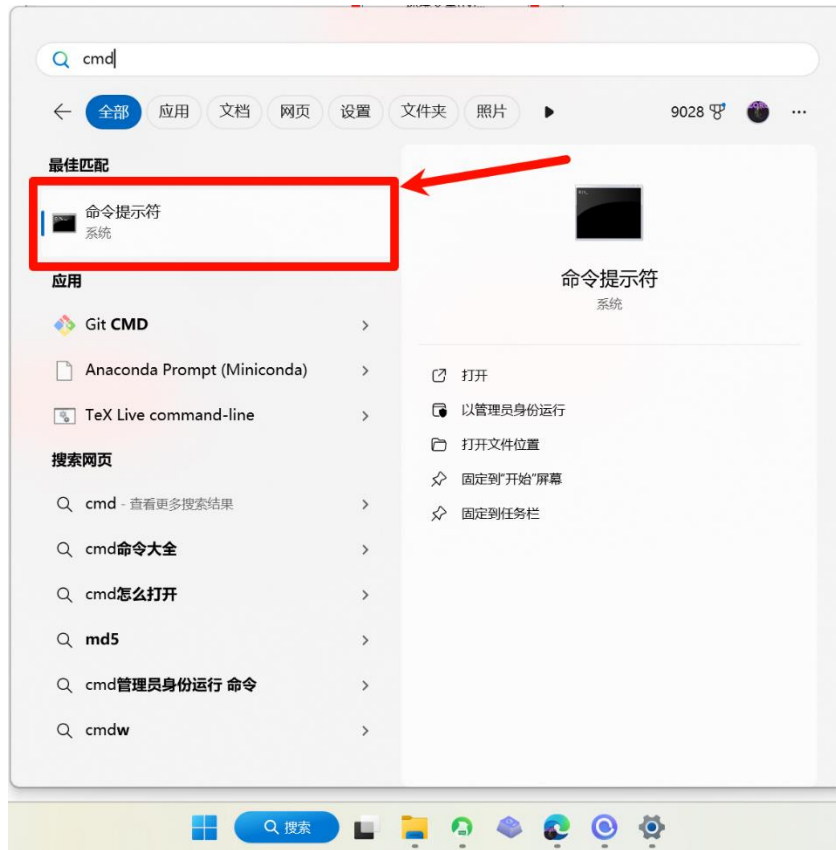
Click "Environment Variables"



Double-click "Path" in the system variables to check if there are three Miniconda file paths as shown in the figure (tick the box for adding PATH in step 7 of software installation, and they will be there)



Input "cmd" in the computer search bar to open the command line tool



Enter the command "`conda --version`" in the command line, and the system will display the "conda version number", indicating successful download.

```
命令提示符
Microsoft Windows [版本 10.0.22631.4037]
(c) Microsoft Corporation。保留所有权利。

C:\Users\zoro>conda --version
conda 24.7.1

C:\Users\zoro>
```

After downloading miniconda, the next step is to download the corresponding versions of cuda, cudnn, and pytorch, as well as set up the pytorch environment.

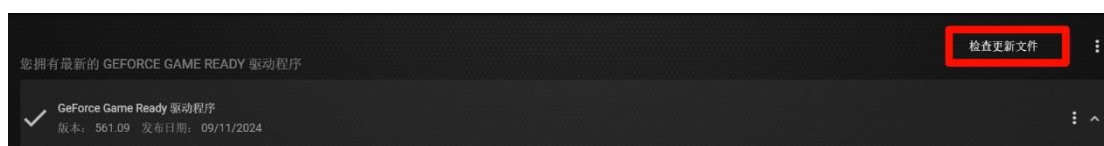
1.2 Pytorch

PyTorch (Python Torch) is an open-source machine learning library primarily used for deep learning tasks. Developed by Facebook's artificial intelligence research group, it provides a flexible tensor data structure and powerful deep learning tools.

In universities, research is generally conducted using the environment provided by the PyTorch library. Therefore, setting up this environment is very helpful for future learning!

(1) CUDA

Before installation, you can upgrade the graphics card driver of your computer. This will not be further explained here (optional step)



Enter the command "nvidia-smi" in the command line to check the CUDA version. The highest supported CUDA version for the blogger is 12.6. CUDA versions are backward compatible, so you can choose to install any CUDA version lower than 12.6

```
C:\Users\zoro>nvidia-smi
Thu Sep 26 10:19:25 2024

+-----+
| NVIDIA-SMI 561.09                  Driver Version: 561.09          CUDA Version: 12.6          |
+-----+-----+
| GPU   Name                               Driver-Model  Bus-Id        Disp.A    Volatile Uncorr. ECC      |
| Fan  Temp  Perf              Pwr:Usage/Cap       |      Id           Memory-Usage  GP  Util    Compute M.  |
|=====+=====+
|  0  NVIDIA GeForce RTX 4060 ... WDDM          00000000:01:00.0 Off      0MiB /  8188MiB      0%      Default  |
| N/A   38C   P8               1W / 130W          |                  |              |
+-----+-----+

Processes:
GPU   GI   CI           PID   Type   Process name                      GPU Memory
  ID   ID   ID                                     Usage
+-----+-----+
No running processes found
+-----+-----+
```

(2) Software installation

Input the following web link: [CUDA Toolkit Archive | NVIDIA Developer](#)

Open the link, select the corresponding CUDA version for your computer, click on Toolkit, and prepare to download the installation package.

Archived Releases

[CUDA Toolkit 12.6.0](#) (August 2024), [Versioned Online Documentation](#)
[CUDA Toolkit 12.5.1](#) (July 2024), [Versioned Online Documentation](#)
[CUDA Toolkit 12.5.0](#) (May 2024), [Versioned Online Documentation](#)
[CUDA Toolkit 12.4.1](#) (April 2024), [Versioned Online Documentation](#)
[CUDA Toolkit 12.4.0](#) (March 2024), [Versioned Online Documentation](#)
[CUDA Toolkit 12.3.2](#) (January 2024), [Versioned Online Documentation](#)
[CUDA Toolkit 12.3.1](#) (November 2023), [Versioned Online Documentation](#)
[CUDA Toolkit 12.3.0](#) (October 2023), [Versioned Online Documentation](#)
[CUDA Toolkit 12.2.2](#) (August 2023), [Versioned Online Documentation](#)
[CUDA Toolkit 12.2.1](#) (July 2023), [Versioned Online Documentation](#)
[CUDA Toolkit 12.2.0](#) (June 2023), [Versioned Online Documentation](#)
[CUDA Toolkit 12.1.1](#) (April 2023), [Versioned Online Documentation](#)
[CUDA Toolkit 12.1.0](#) (February 2023), [Versioned Online Documentation](#)
[CUDA Toolkit 12.0.1](#) (January 2023), [Versioned Online Documentation](#)
[CUDA Toolkit 12.0.0](#) (December 2022), [Versioned Online Documentation](#)
[CUDA Toolkit 11.8.0](#) (October 2022), [Versioned Online Documentation](#)
[CUDA Toolkit 11.7.1](#) (August 2022), [Versioned Online Documentation](#)
[CUDA Toolkit 11.7.0](#) (May 2022), [Versioned Online Documentation](#)
[CUDA Toolkit 11.6.2](#) (March 2022), [Versioned Online Documentation](#)
[CUDA Toolkit 11.6.1](#) (February 2022), [Versioned Online Documentation](#)
[CUDA Toolkit 11.6.0](#) (January 2022), [Versioned Online Documentation](#)
[CUDA Toolkit 11.5.2](#) (February 2022), [Versioned Online Documentation](#)
[CUDA Toolkit 11.5.1](#) (November 2021), [Versioned Online Documentation](#)
[CUDA Toolkit 11.5.0](#) (October 2021), [Versioned Online Documentation](#)
[CUDA Toolkit 11.4.4](#) (February 2022), [Versioned Online Documentation](#)
[CUDA Toolkit 11.4.3](#) (November 2021), [Versioned Online Documentation](#)
[CUDA Toolkit 11.4.2](#) (September 2021), [Versioned Online Documentation](#)
[CUDA Toolkit 11.4.1](#) (August 2021), [Versioned Online Documentation](#)
[CUDA Toolkit 11.4.0](#) (June 2021), [Versioned Online Documentation](#)
[CUDA Toolkit 11.3.1](#) (May 2021), [Versioned Online Documentation](#)
[CUDA Toolkit 11.3.0](#) (April 2021), [Versioned Online Documentation](#)



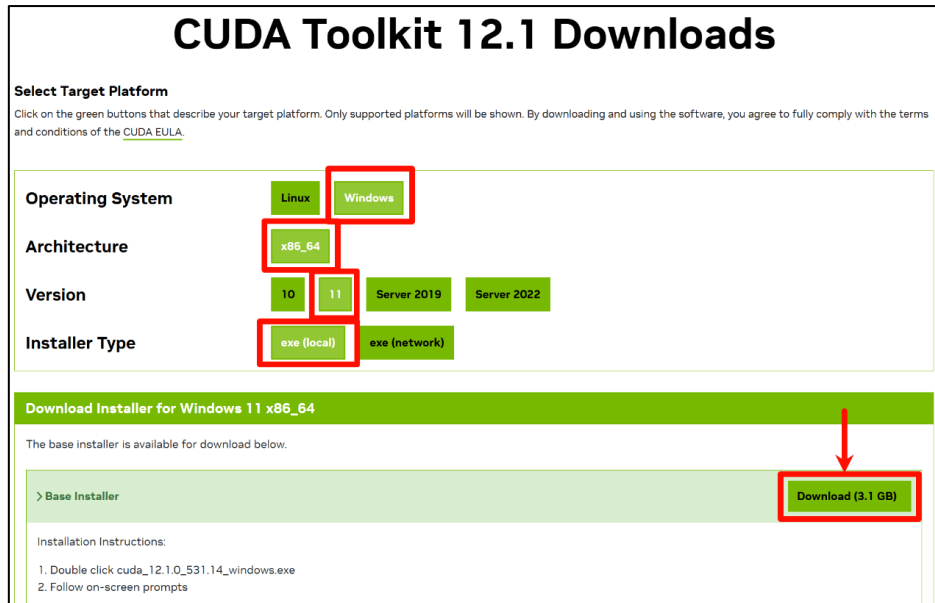
选择自己电脑对应的CUDA版本后
点击进行安装

After selecting according to the computer version, click "Download" to download the software installation package.

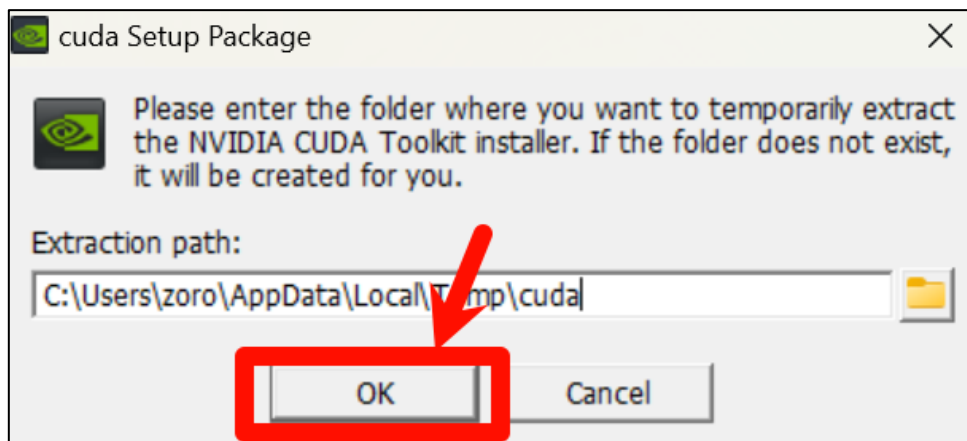
Note: (1) For Windows 10 system, select 10 in the <Version> column,

(2) If this page cannot be opened, you can try using a VPN,

(3) The download time for the software package is relatively long. Please be patient.



Open the installation package, click "OK", and start the CUDA download (the path here can be left as default)



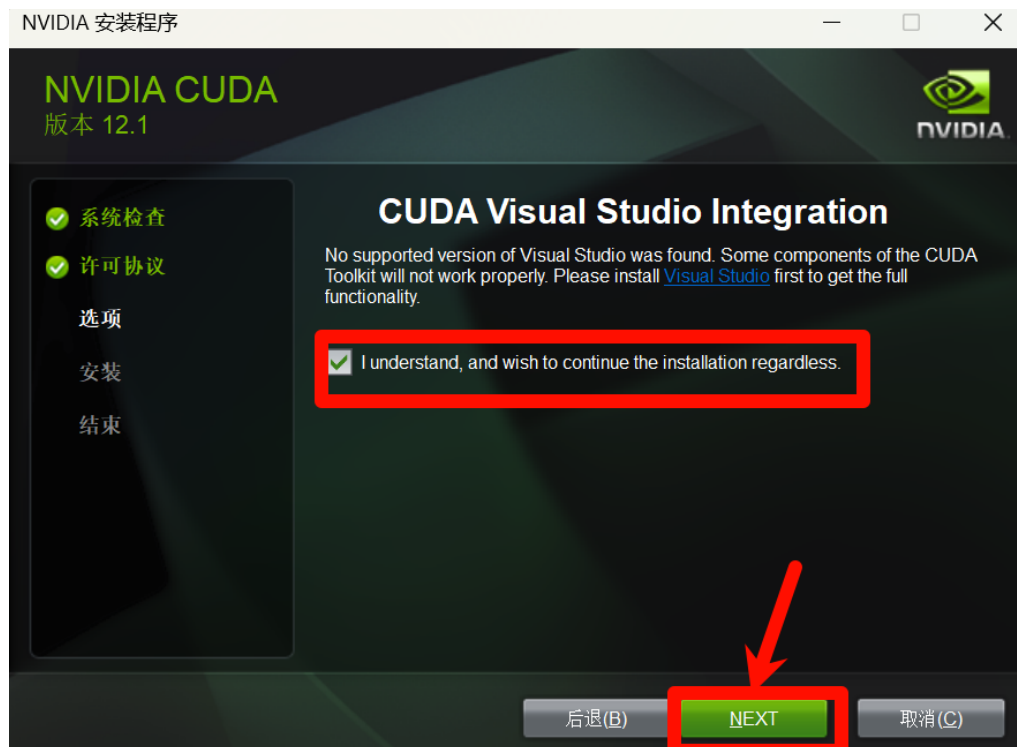
After the unzipping is completed, the system checks the compatibility of the installation program with the system, and then waits for a moment



Select "Minimal" installation and click "Next"



Agree to the agreement, click "Next", and start the installation

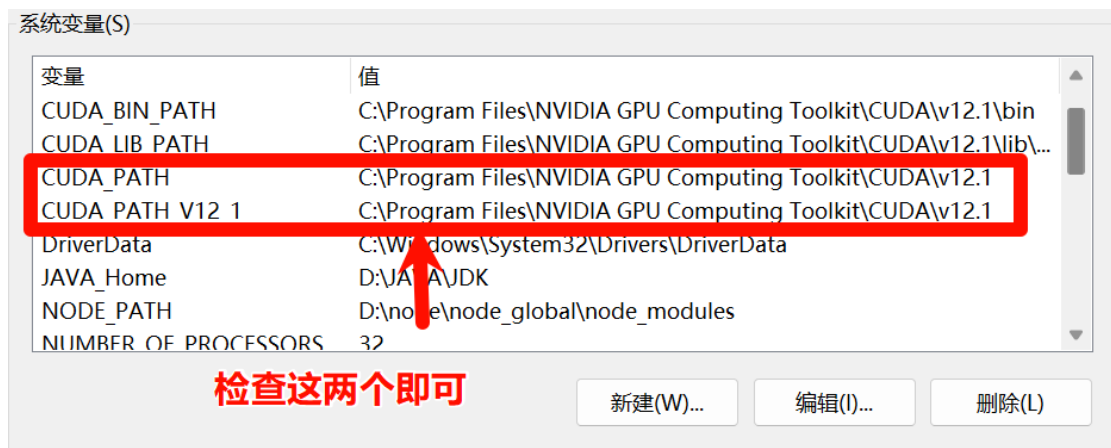


The installer has completed. Click "Close"

(3) Environment Verification

Open environment variables in settings

Check if there are two CUDA paths as shown in the figure



Enter "nvcc --version" in the command line, and if the system displays as shown below, it indicates successful installation

```
命令提示符
Microsoft Windows [版本 10.0.22631.4037]
(c) Microsoft Corporation。保留所有权利。

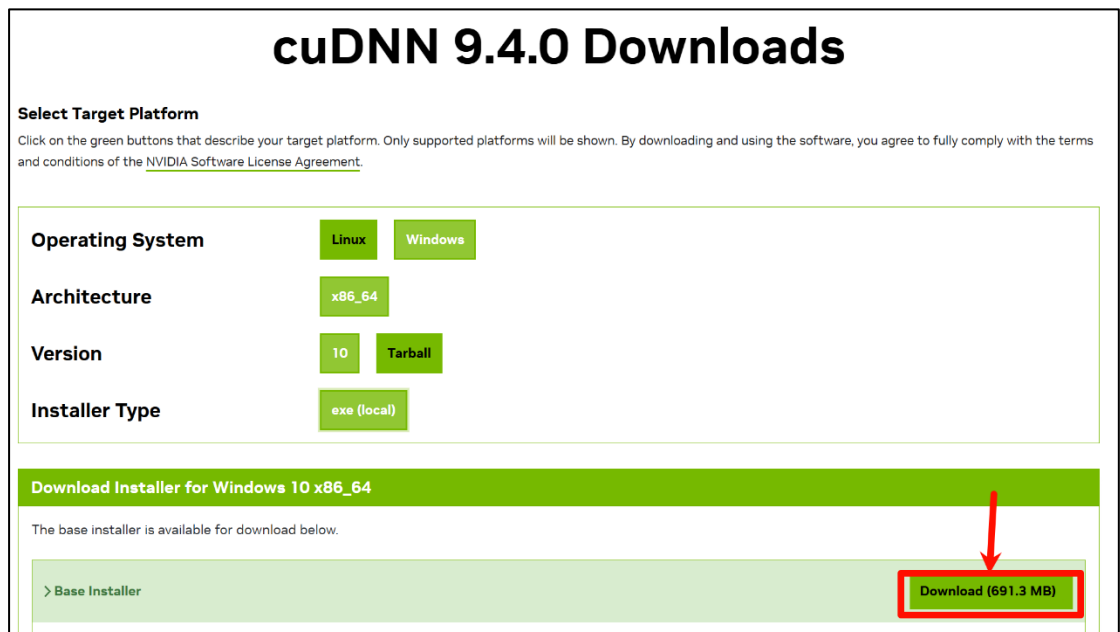
C:\Users\zoro>nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2023 NVIDIA Corporation
Built on Wed_Feb__8_05:53:42_Coordinated_Universal_Time_2023
Cuda compilation tools, release 12.1, V12.1.66
Build cuda_12.1.r12.1/compiler.32415258_0

C:\Users\zoro>
```

1.3 CUDNN

When downloading the version number of cudnn, it is important to ensure that it corresponds to the version number of cuda

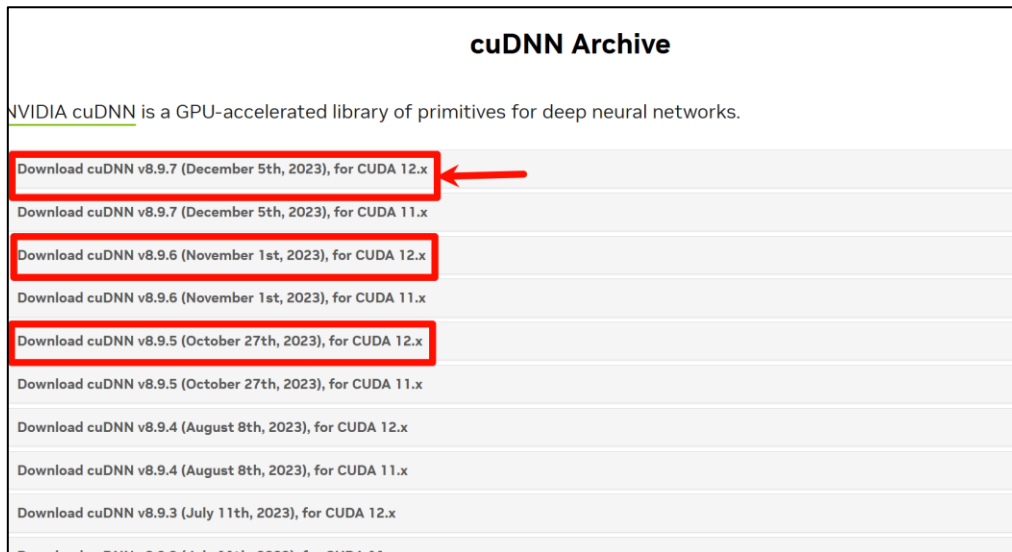
The graphics card of our research group is a 4060. Taking this as an example, readers can directly download the installation package from [cuDNN 9.4.0 Downloads | NVIDIA Developer](#) and install it following the CUDA installation method.



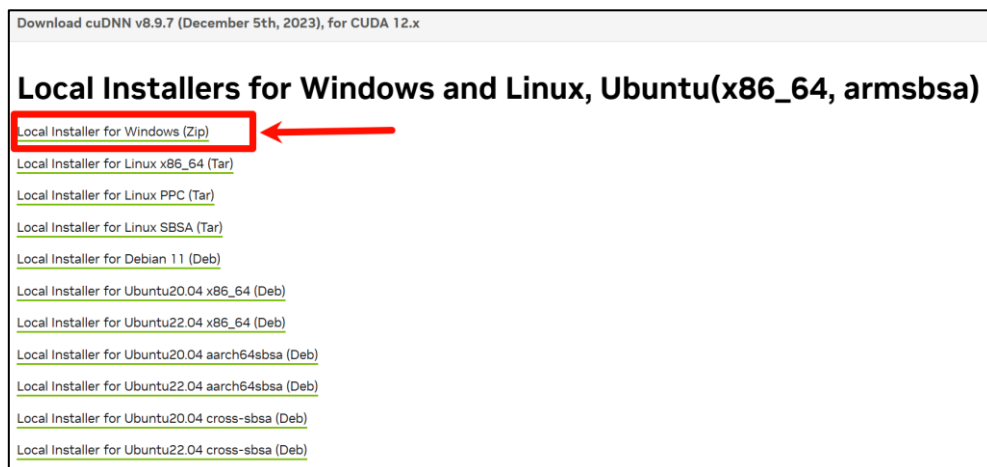
(1) Software installation.

Download link: [cuDNN Archive | NVIDIA Developer](#)

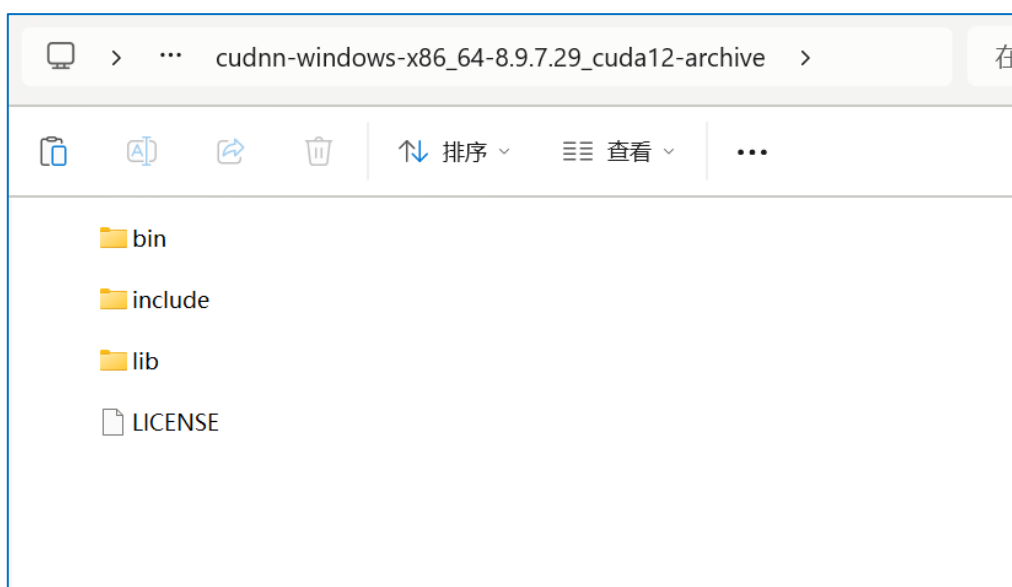
Enter the website and find the corresponding version of CUDNN (for example, if the team downloaded CUDA version 12.1, select the version for CUDA 12.x to download)



Select the first option and click to download (Cudnn is a zip file)

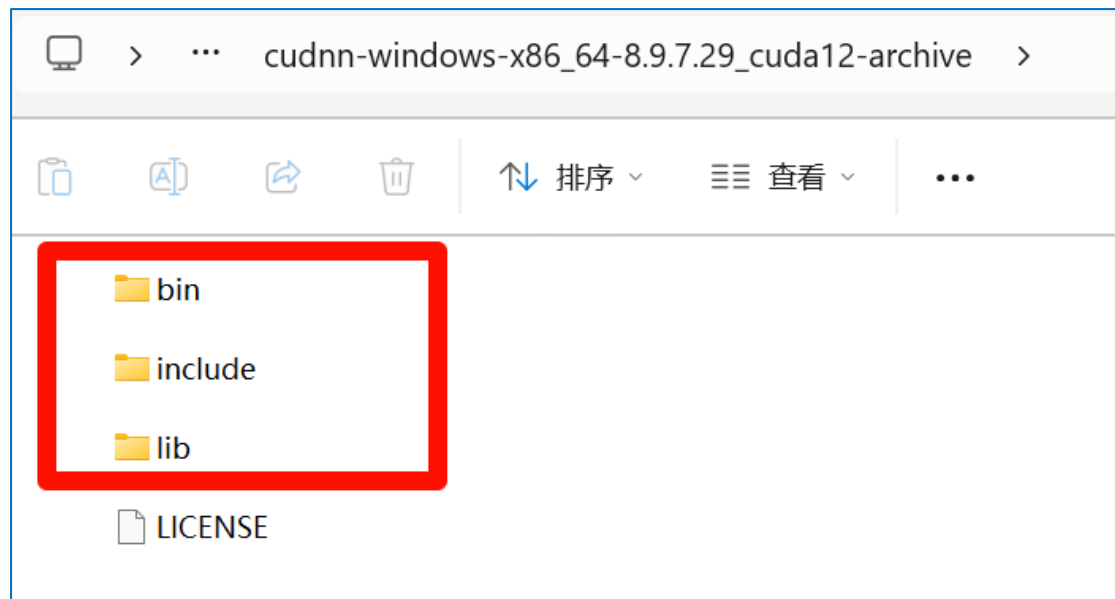


After decompressing the ZIP file, open the folder as follows



Simply copy and paste these three files into the CUDA folder location (the path can be found

in the environment variables)



Check and verify the CUDA folder path in the environment variables

CUDA_PATH	C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.1
CUDA_PATH_V12_1	C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.1

After completing the software installation, readers can use conda to set up the pytorch environment

(2) environment setup

Open the command line tool, and the reader enters "conda create -n pytorch python=3.8" to create a new Python environment and names it pytorch

```
msys2-conda-epoch conda-forge/win-64::msys2-conda-epoch-20160416-1
openssl conda-forge/win-64::openssl-3.3.2-h2466b09_0
pip conda-forge/noarch::pip-24.2-pyh8b19718_1
pthread-stubs conda-forge/win-64::pthread-stubs-0.4-hcd874cb_1001
python conda-forge/win-64::python-3.8.20-hfaddaf0_1_cpython
setuptools conda-forge/noarch::setuptools-75.1.0-pyhd8ed1ab_0
tk conda-forge/win-64::tk-8.6.13-h5226925_1
ucrt conda-forge/win-64::ucrt-10.0.22621.0-h57928b3_0
vc conda-forge/win-64::vc-14.3-h8a93ad2_21
vc14_runtime conda-forge/win-64::vc14_runtime-14.40.33810-ha82c5b3_21
vs2015_runtime conda-forge/win-64::vs2015_runtime-14.40.33810-h3bf8584_21
wheel conda-forge/noarch::wheel-0.44.0-pyhd8ed1ab_0
xorg-kbproto conda-forge/win-64::xorg-kbproto-1.0.7-hcd874cb_1002
xorg-libx11 conda-forge/win-64::xorg-libx11-1.8.9-h0076a8d_1
xorg-libxau conda-forge/win-64::xorg-libxau-1.0.11-hcd874cb_0
xorg-libxdmcp conda-forge/win-64::xorg-libxdmcp-1.1.3-hcd874cb_0
xorg-xextproto conda-forge/win-64::xorg-xextproto-7.3.0-hcd874cb_1003
xorg-xproto conda-forge/win-64::xorg-xproto-7.0.31-hcd874cb_1007
xz conda-forge/win-64::xz-5.2.6-h8d14728_0

Proceed ([y]/n)? y
```

Enter "y" in the "proceed" field, and the author agrees to the download. The system will display the message "done", indicating that the download is complete.


```

libxcb          conda-forge/win-64::libxcb-1.16-h013a479_1
libzlib         conda-forge/win-64::libzlib-1.3.1-h2466b09_1
m2w64-gcc-libgfor~ conda-forge/win-64::m2w64-gcc-libgfortran-5.3.0-6
m2w64-gcc-libs  conda-forge/win-64::m2w64-gcc-libs-5.3.0-7
m2w64-gcc-libs-co~ conda-forge/win-64::m2w64-gcc-libs-core-5.3.0-7
m2w64-gmp       conda-forge/win-64::m2w64-gmp-6.1.0-2
m2w64-libwinpthre~ conda-forge/win-64::m2w64-libwinpthread-git-5.0.0.4634.697f757-2
msys2-conda-epoch conda-forge/win-64::msys2-conda-epoch-20160418-1
openssl         conda-forge/win-64::openssl-3.3.2-h2466b09_0
pip             conda-forge/noarch::pip-24.2-pyh8b19718_1
pthread-stubs   conda-forge/win-64::pthread-stubs-0.4-hcd874cb_1001
python          conda-forge/win-64::python-3.8.20-hfaddaf0_1_cpython
setuptools      conda-forge/noarch::setuptools-75.1.0-pyhd8ed1ab_0
tk              conda-forge/win-64::tk-8.6.13-h5226925_1
ucrt            conda-forge/win-64::ucrt-10.0.22621.0-h57928b3_0
vc              conda-forge/win-64::vc-14.3-h8a93ad2_21
vc14_runtime    conda-forge/win-64::vc14_runtime-14.40.33810-ha82c5b3_21
vs2015_runtime  conda-forge/win-64::vs2015_runtime-14.40.33810-h3bf8584_21
done
#
# To activate this environment, use
#
#   $ conda activate pytorch
#
# To deactivate an active environment, use
#
#   $ conda deactivate

```

Continue to enter "conda activate pytorch" in the command line. At this point, the left side of the system will display "pytorch", indicating that the system installation is successful.

```

C:\Users\zoro>conda activate pytorch
(pytorch) C:\Users\zoro>

```

Authors can install conda via the command line. To install both conda and pip, including the installation of domestic mirrors in China, run the following two lines of code.

- 1) `.pip config set install.trusted-host mirrors.aliyun.com`
- 2) `.conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/`

Navigate to the PyTorch official website, scroll down to the homepage, select the corresponding version of PyTorch, and copy the command into the command line (the GPU version requires scientific internet access, and domestic sources can only download the CPU version).

PyTorch Build	Stable (2.4.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.1	CUDA 12.4	CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121</pre>			

复制这段指令到命令行中

The copied command is as follows:

`pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121`

Simultaneously, enter the following command in the command line:

`"torch.cuda.is_available()"` checks whether the torch version supports GPU. If it returns True, it indicates successful installation

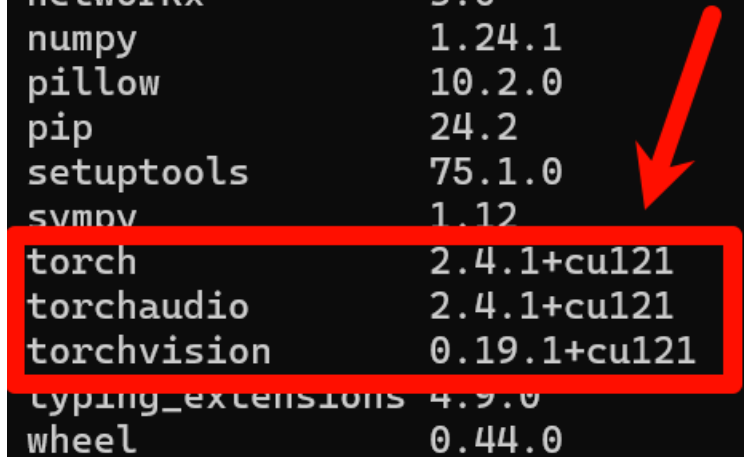
Input the command copied in the previous step to install the PyTorch library

```
(pytorch) C:\Users\zoro>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
Looking in indexes: https://download.pytorch.org/whl/cu121
Requirement already satisfied: torch in d:\miniconda3\envs\pytorch\lib\site-packages (2.4.1+cu121)
Requirement already satisfied: torchvision in d:\miniconda3\envs\pytorch\lib\site-packages (0.19.1+cu121)
Requirement already satisfied: torchaudio in d:\miniconda3\envs\pytorch\lib\site-packages (2.4.1+cu121)
Requirement already satisfied: filelock in d:\miniconda3\envs\pytorch\lib\site-packages (from torch) (3.13.1)
Requirement already satisfied: typing-extensions>=4.8.0 in d:\miniconda3\envs\pytorch\lib\site-packages (from torch) (4.9.0)
Requirement already satisfied: sympy in d:\miniconda3\envs\pytorch\lib\site-packages (from torch) (1.12)
Requirement already satisfied: networkx in d:\miniconda3\envs\pytorch\lib\site-packages (from torch) (3.0)
```

(3) Environmental verification

Enter "pip list" to check the version number of PyTorch and confirm that the PyTorch environment has been successfully configured

```
(pytorch) C:\Users\zoro>pip list
Package              Version
-----
filelock             3.13.1
fsspec               2024.2.0
Jinja2               3.1.3
MarkupSafe           2.1.5
mpmath               1.3.0
networkx             3.0
numpy                1.24.1
pillow               10.2.0
pip                  24.2
setuptools           75.1.0
sympy                1.12
torch                 2.4.1+cu121
torchaudio           2.4.1+cu121
torchvision          0.19.1+cu121
typing_extensions    4.9.0
wheel                 0.44.0
```



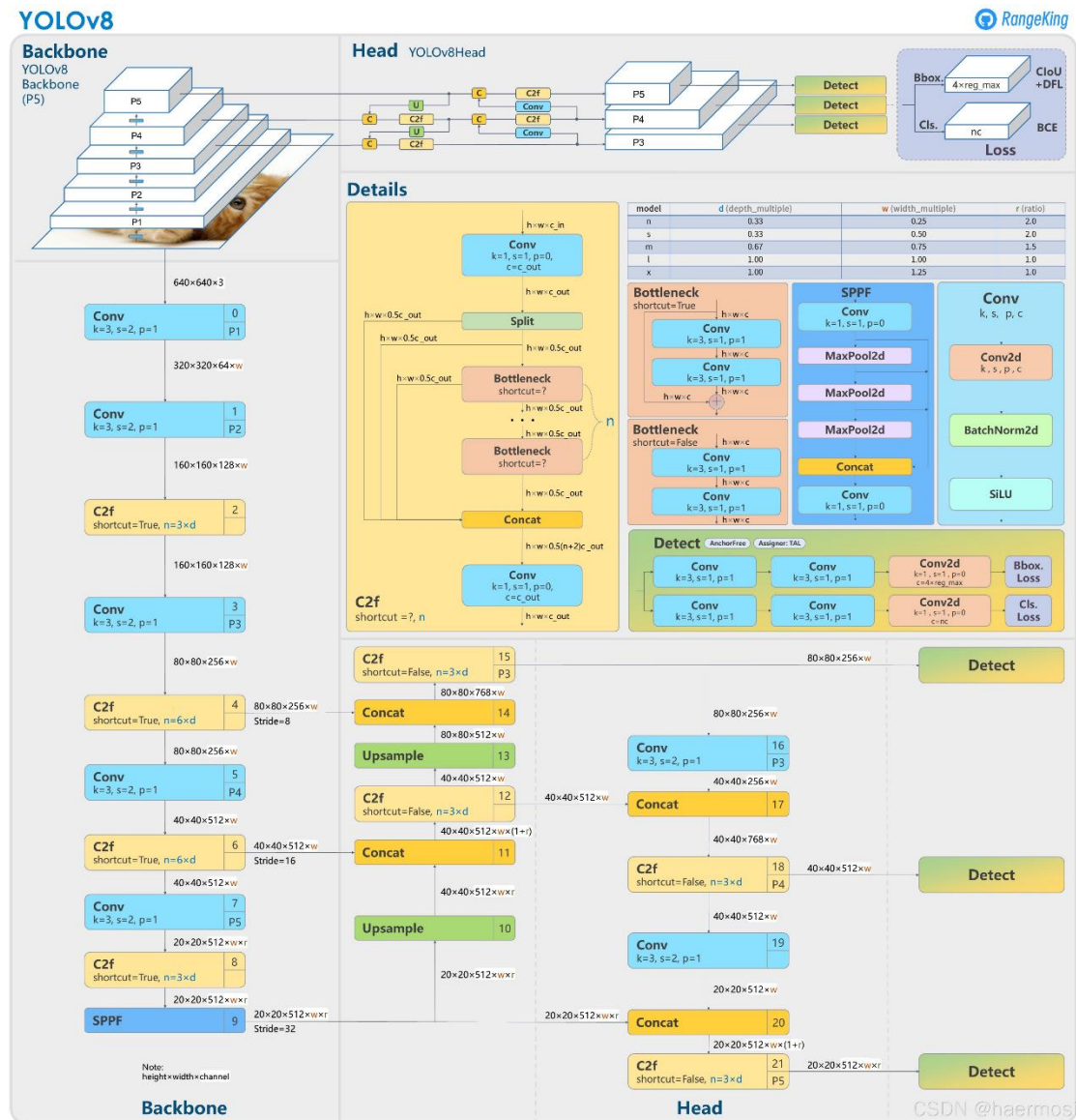
At this point, a commonly used PyTorch environment has been set up. After the environment is configured, the author explains how to apply the PyTorch environment to the YOLOv8 project.

II YOLOv8

Yolov8 is the latest Yolo series object detection algorithm launched by Ultralytics, which can be used for tasks such as image classification, object detection, and instance segmentation. There are many ways to download and configure YOLOv8. It is recommended to directly download the source code from GitHub/Gitee and open it.

1. Installation of the basic model of yolov8

The YOLOv8 network structure is shown below (sourced from GitHub user RangeKing)

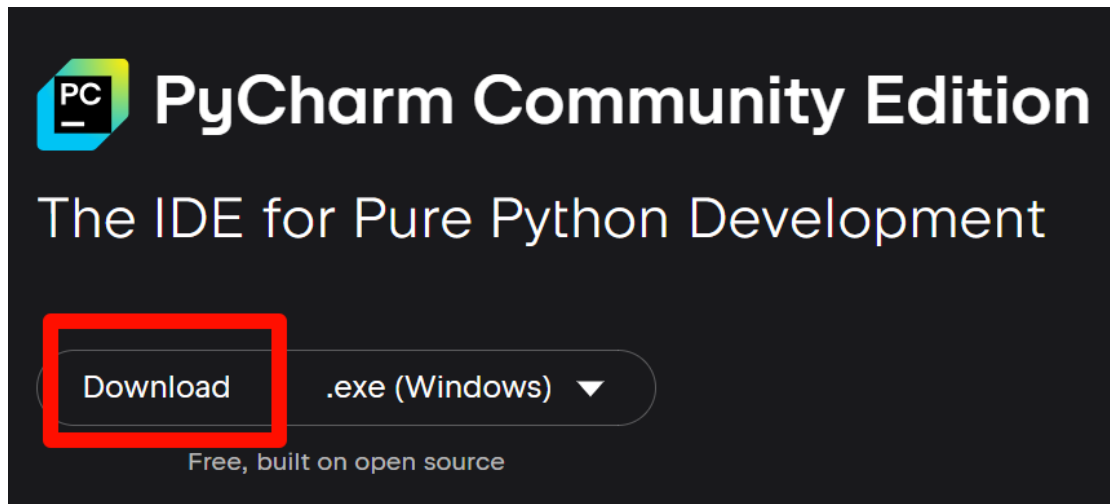


1.1 Pycharm

Pycharm is a compiler for Python code. We can open and modify Python code in this software.

(1) Download

The download link for the software is: [Download PyCharm: The Python IDE for data science and web development by JetBrains](#), Click the above link to enter the official website of Pycharm and find the community version to download (do not download the professional version, which is paid)



After downloading, open the installation package and click "Next" button



After selecting the installation path for the software and click the "Next" button



After selecting the installation path for the software, click the "Next" button.



Keep the default "JetBrains" folder in the menu directory, and click "Install"

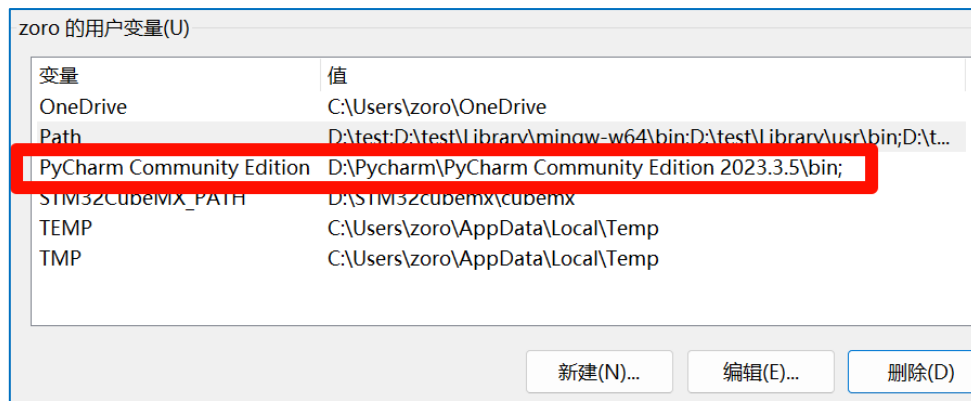


Click the "Finish" button and choose whether to restart the computer immediately or later, depending on your situation.



(2) Environment Verification

Open the environment variables and check if Pycharm is present in the user variables



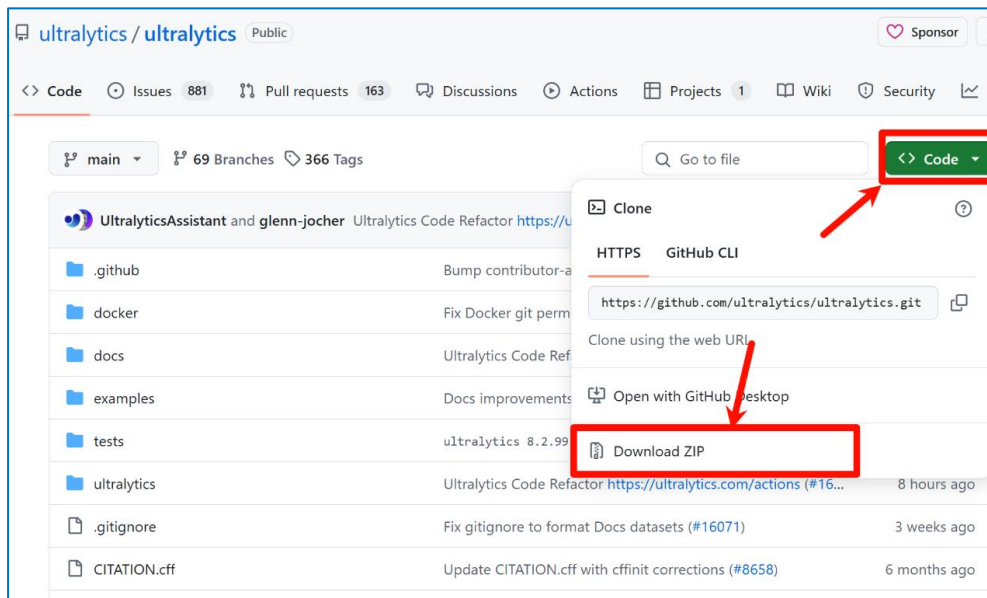
1.2 Download, verification, and usage of YOLOv8

(1) Environment setup

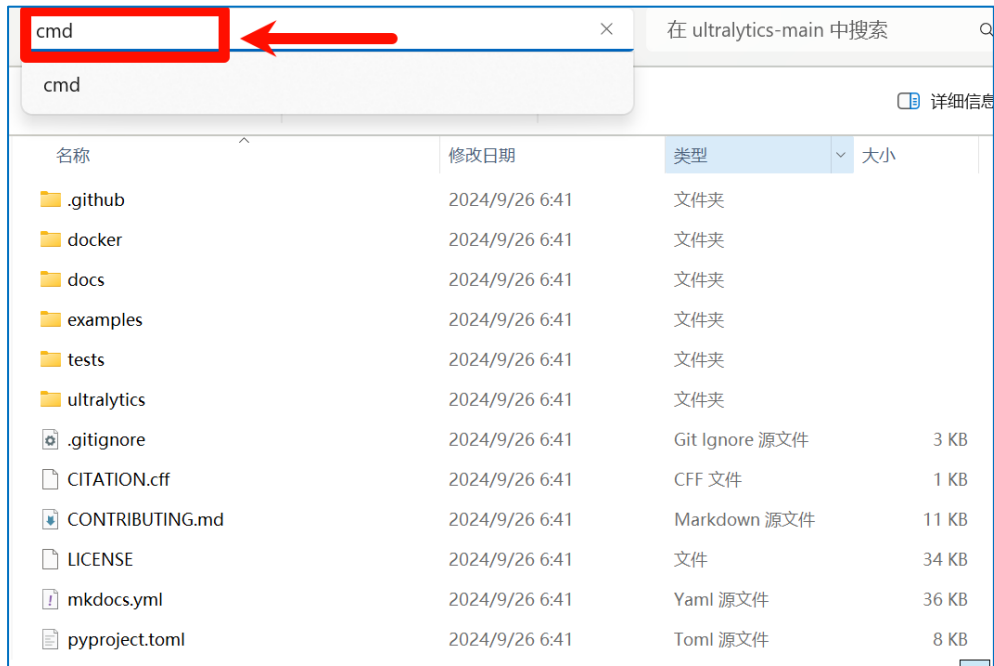
Download link: [Github: haermosi/yolov8: YOLOv8](https://github.com/haermosi/yolov8)

<https://github.com/ultralytics/ultralytics>

Click "Code", then click "Download" to download the zip file



After decompressing the file package, enter "cmd" in this folder path to open the command line



Enter "conda activate pytorch" in the command line to activate the previously configured environment. Then, continue to enter "pip install -e ." to download the relevant libraries

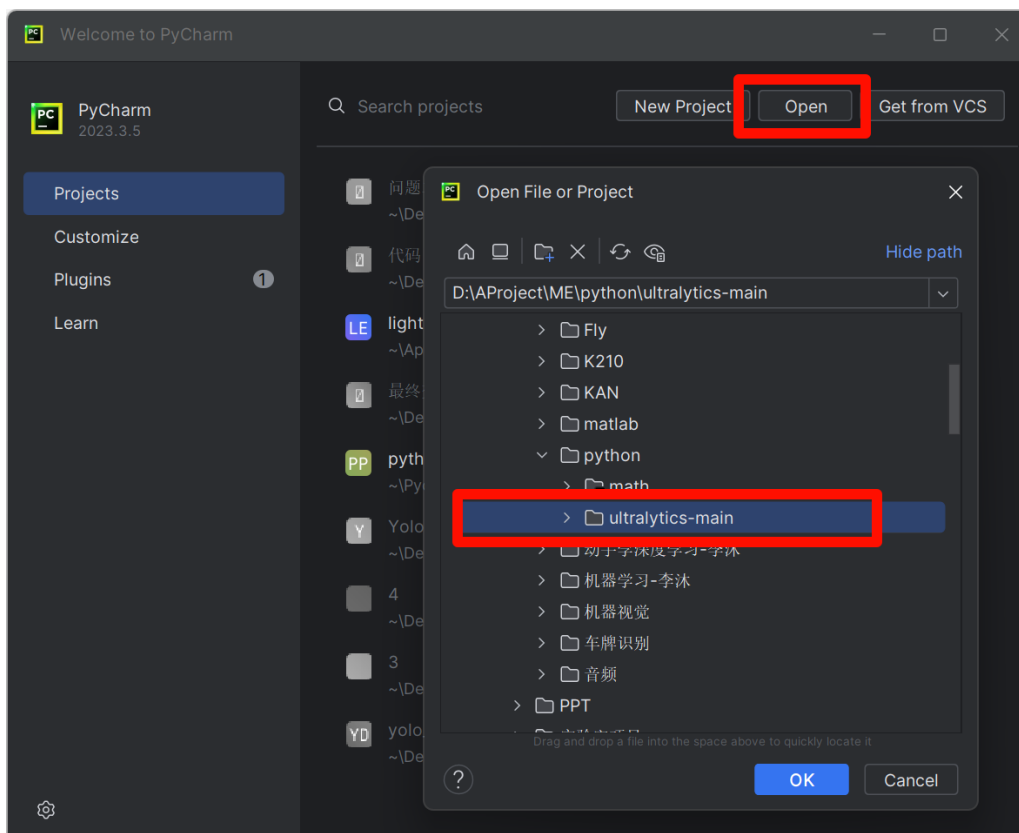
```
(pytorch) D:\AProject\ME\python\ultralytics-main>pip install -e .
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Obtaining file:///D:/AProject/ME/python/ultralytics-main
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Requirement already satisfied: numpy<2.0.0,>=1.23.0 in d:\miniconda3\envs\pytorch (1.24.1)
Collecting matplotlib>=3.3.0 (from ultralytics==8.2.101)
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/16/51/58b0b9de.../matplotlib-3.7.5-cp38-cp38-win_amd64.whl (7.5 MB)
Collecting opencv-python>=4.6.0 (from ultralytics==8.2.101)
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/ec/6c/fab81134.../opencv-python-4.10.0.84-cp37-abi3-win_amd64.whl (38.8 MB)
Requirement already satisfied: pillow>=7.1.2 in d:\miniconda3\envs\pytorch (9.4.0)
Collecting pyyaml>=5.3.1 (from ultralytics==8.2.101)
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/75/8a/ee831ad5f.../pyyaml-6.0.2-cp38-cp38-win_amd64.whl (162 kB)
Collecting requests>=2.23.0 (from ultralytics==8.2.101)
  Using cached https://pypi.tuna.tsinghua.edu.cn/packages/f9/9b/335f9764.../requests-2.32.3-py3-none-any.whl (64 kB)
```

Environmental verification and usage

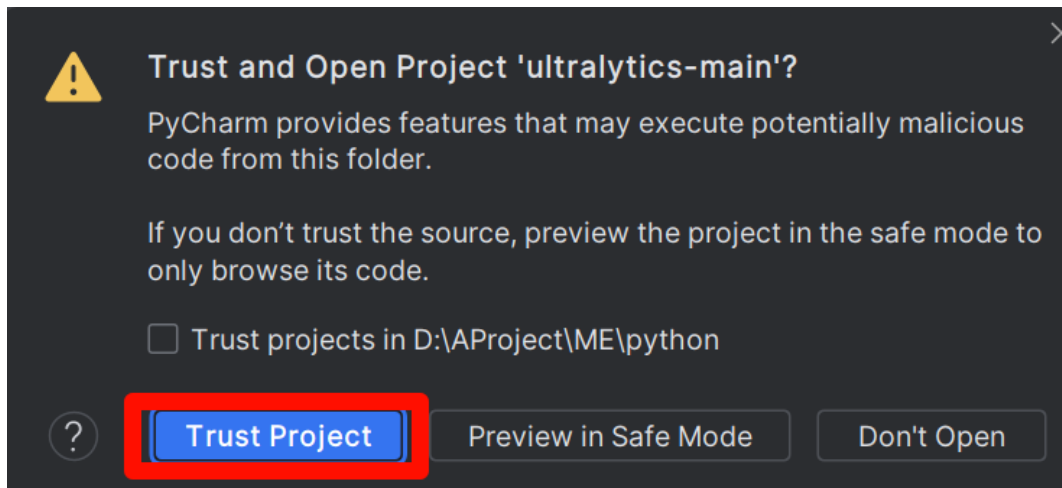
Enter "pip list" to check if the installation is successful. The image below shows that the installation is complete "conda activate pytorch".

```
MarkupSafe      2.1.5
matplotlib      3.7.5
mpmath           1.3.0
networkx        3.0
numpy           1.24.1
opencv-python   4.10.0.84
packaging       24.1
pandas          2.0.3
pillow          10.2.0
pip             24.2
psutil          6.0.0
py-cpuinfo      9.0.0
pyparsing       3.1.4
python-dateutil 2.9.0.post0
pytz            2024.2
PyYAML          6.0.2
requests        2.32.3
scipy           1.10.1
seaborn         0.13.2
setuptools      75.1.0
six             1.16.0
sympy           1.12
torch           2.4.1+cu121
torchaudio      2.4.1+cu121
torchvision     0.19.1+cu121
tqdm            4.66.5
typing_extensions 4.9.0
tzdata          2024.2
ultralitics     8.2.101    D:\AProject\ME\python\ultralitics-main
urllib3         2.2.3
```

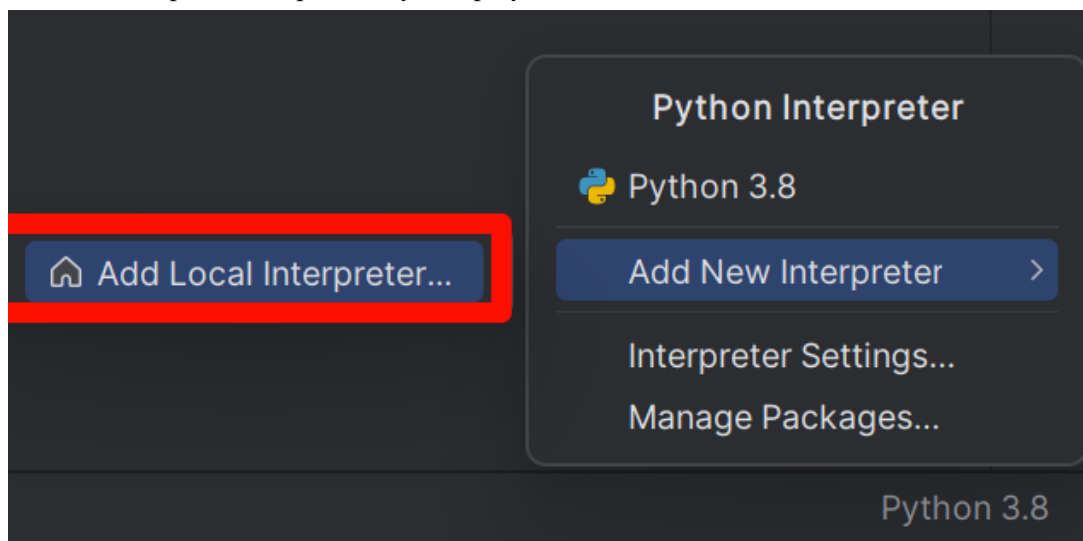
Open PyCharm, click "Open", and navigate to the yolov8 folder



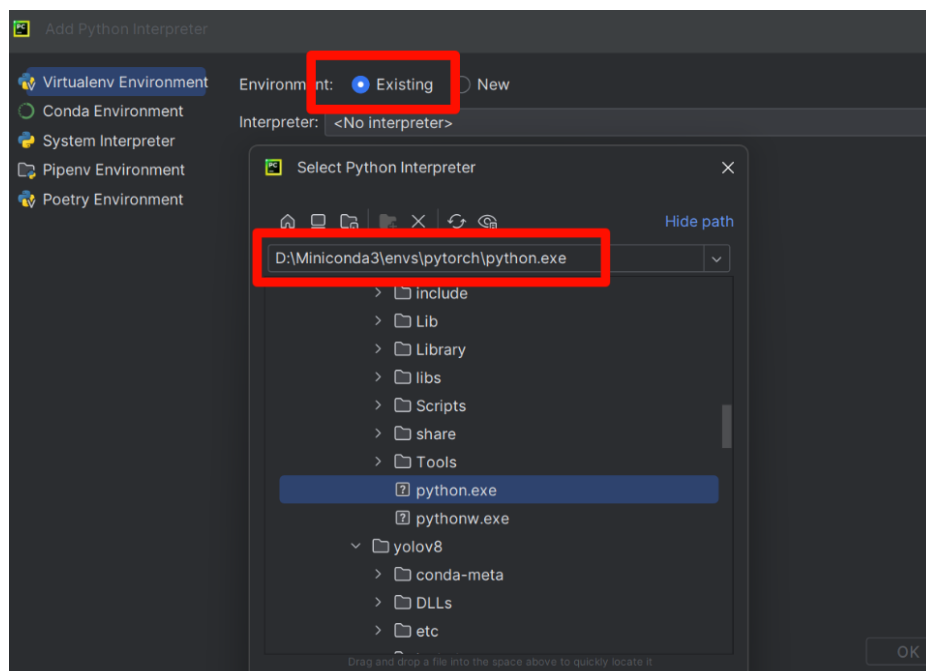
After clicking the OK button, add a new environment in the lower right corner of the pop-up interface

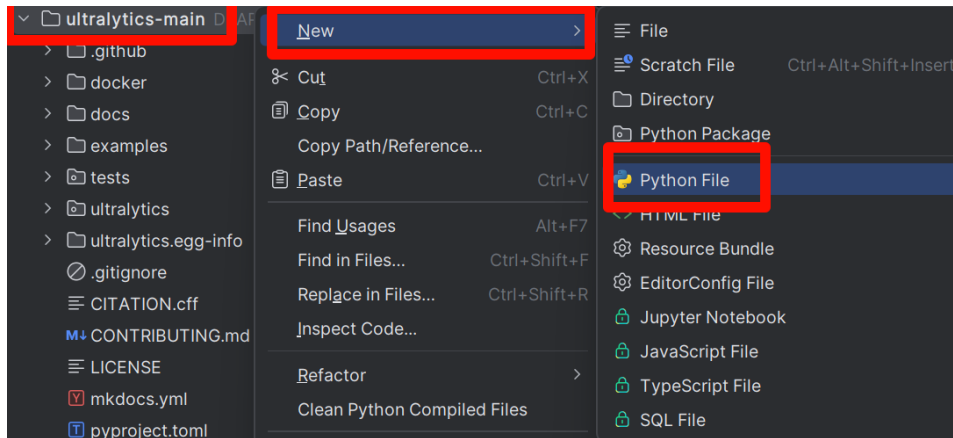


Select the path of the previously set up PyTorch environment



Create a new Python file and give it a name of your own, such as "test.py"



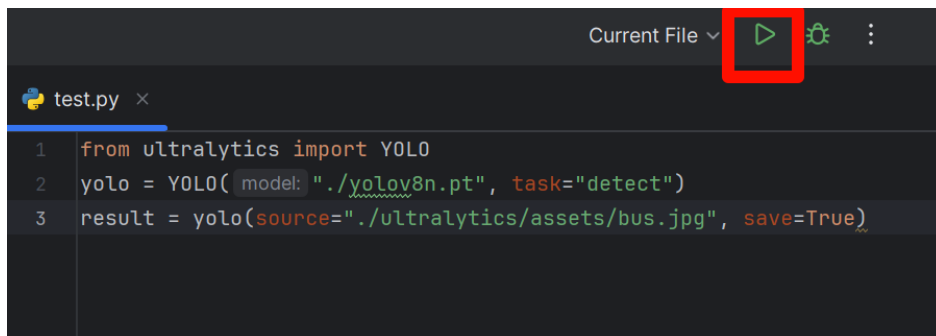


Input the following code and click "Run"

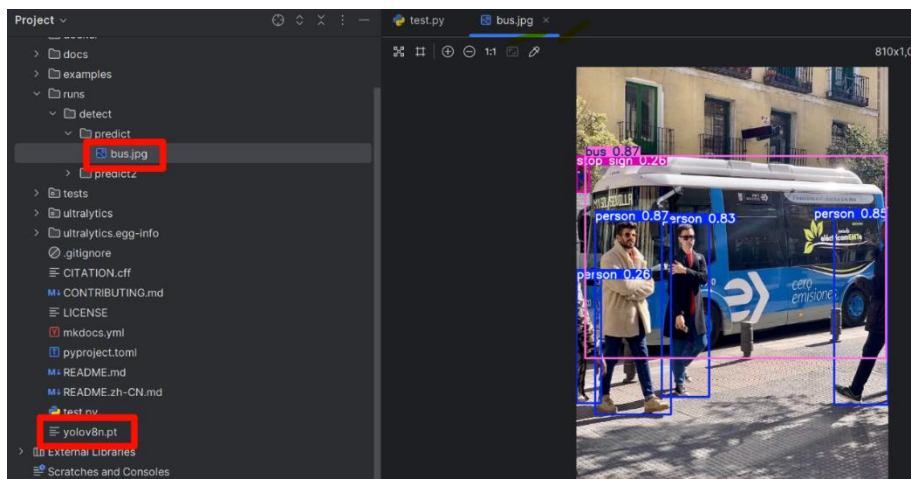
```
from ultralytics import YOLO
```

```
yolo = YOLO("./yolov8n.pt", task="detect")
```

```
result = yolo(source="./ultralytics/assets/bus.jpg", save=True)
```



After the program finishes running, the model training is completed. If I generate a yolov8n.pt and bus.jpg file in the system folder as shown on the right, it indicates that the environment configuration is successful



So far, the setup of the YOLOv8 environment and model training have been basically completed, with the remaining work being model improvement.

1.3 Verify whether the GPU is available

After the installation is complete, verify whether PyTorch can utilize the GPU (YOLOv8 performs best when based on PyTorch): therefore, enter the following code in the terminal:

```
“# 进入 Python 交互环境
python
# 执行以下代码
import torch
import ultralytics
from ultralytics import YOLO
# 检查 PyTorch GPU 支持
print("PyTorch GPU 可用: ", torch.cuda.is_available())
# 检查 YOLOv8 GPU 支持
print("YOLOv8 GPU 设备: ", YOLO("yolov8n.pt").device)
# 退出 Python 交互环境“
```

If the output is "PyTorch GPU available: True" and "YOLOv8 GPU device: cuda:0", it indicates that the GPU configuration is successful; if "cpu" is displayed, you need to check whether the CUDA/cuDNN installation is correct.

III. Using GPU to Train YOLOv8 Model (Practical)

Taking a subset of the COCO dataset as an example, we demonstrate the entire GPU training process. You can replace it with your own dataset.

1. Improvement of the yolov8 model

1.1 Prepare the dataset

(1) Dataset format:

YOLOv8 requires the dataset to be in YOLO format, with the directory structure as follows:

Plaintext

```
my_dataset/
├── images/          # 图片文件夹
│   ├── train/      # 训练集图片
│   └── val/         # 验证集图片
└── labels/         # 标签文件夹
    ├── train/      # 训练集标签 (.txt 格式)
    └── val/        # 验证集标签 (.txt 格式)
```

(2) Creating configuration files:

Create a dataset.yaml file in the dataset directory, with the following content:

Yaml

```
path: ./my_dataset # 数据集根路径
train: images/train # 训练集图片路径
val: images/val     # 验证集图片路径
nc: 2               # 类别数 (示例: 人、车)
names: ['people', 'car'] # 类别名称
```

2.2 Initiate GPU training

Start training using the official YOLOv8 API, specifying the GPU device:

```
# train_yolov8.py
from ultralytics import YOLO
# 加载预训练模型 (n/s/m/l/x, n 最轻量, x 精度最高)
model = YOLO("yolov8n.pt")
# 启动 GPU 训练
results = model.train(
    data="my_dataset/dataset.yaml", # 数据集配置文件路径
    epochs=100,                     # 训练轮数
    batch=16,                       # 批量大小 (根据 GPU 显存调整, 显存小则调小)
    device=0,                       # 指定 GPU 设备 (0 为第一张显卡, 多卡可设[0,1])
    imgsz=640,                      # 输入图片尺寸
    lr=0.01,                        # 初始学习率
    save=True,                      # 保存训练结果
    project="runs/train",           # 训练结果保存路径
    name="my_yolov8_model"         # 模型名称
)
# 验证模型
metrics = model.val()              # 用验证集评估模型
print("mAP@0.5:", metrics.box.map50) # 输出 mAP 指标
```

3.3 Training via command line (more concise)

Start training using the official YOLOv8 API, specifying the GPU device:

```
# GPU 训练命令
yolo detect train \
    data=my_dataset/dataset.yaml \
    model=yolov8n.pt \
    epochs=100 \
    batch=16 \
    device=0 \
    imgsz=640 \
    project=runs/train \
    name=my_yolov8_model
```

3.4 Monitoring the Training Process

Real-time log: The terminal will output metrics such as loss, mAP, and FPS for each training round. If the FPS is much higher than that of the CPU (e.g., GPU FPS=50+, CPU FPS=2-3), it indicates that the GPU is working;

Visualization monitoring: After training is completed, the results will be saved in the "runs/train/my_yolov8_model" directory, including:

- weights/best.pt: Optimal model weights;
- results.png: Training loss/accuracy curve;

val_batch0_pred.jpg: Visualization results of predictions on the validation set.

3.5 Common GPU Training Issues

(1) Error "CUDA out of memory": GPU memory is insufficient. Solution:

Reduce the batch size (e.g., from 16 to 8/4);

Reduce imgsz (e.g. from 640 to 480);

Use a more lightweight model (such as yolov8n.pt instead of yolov8x.pt);

(2) Error message: "no CUDA-capable device is detected": GPU not recognized. Solution:

Check whether the graphics card driver is installed correctly;

Confirm that the CUDA/cuDNN version matches the driver;

Reactivate the virtual environment after restarting the computer;

(3) Slow training speed (GPU not utilized):

Check whether torch.cuda.is_available() is True;

During training, specify device=0 to avoid using the CPU by default.